Métricas de Engenharia de Software

Conteúdo dos Flashcards

Conteúdo dos Flashcards: Métrica LOC

Flashcard 1.1 - Definição e Propósito: Significa Lines of Code (Linhas de Código). É uma das métricas mais antigas da engenharia de software e serve para medir o tamanho do software contando o número de linhas do seu código-fonte.

Flashcard 1.2 - Formas de Contagem: Existem duas formas principais de contar: Linhas Físicas, que contam absolutamente todas as linhas do arquivo, e Linhas Lógicas, que consideram apenas as instruções reais e comandos que o programa executa.

Flashcard 1.3 - Vantagens de Uso: O seu principal ponto forte é a simplicidade. Ela é muito fácil de medir, sua aplicação tem baixo custo e o processo pode ser totalmente automatizado por ferramentas.

Flashcard 1.4 - Limitações e Problemas: A métrica mede apenas quantidade, não qualidade ou complexidade real. Além disso, o resultado varia muito dependendo da linguagem de programação utilizada e do estilo de escrita de cada desenvolvedor, lembrando que mais linhas de código não significam um software melhor.

Conteúdo dos Flashcards: Métrica Pontos de Função (PF)

Flashcard 2.1 - Definição: É uma métrica que mede o tamanho do software com base nas funcionalidades entregues ao usuário, focando no o que o sistema faz (visão do usuário) e ignorando como ele é implementado.

Flashcard 2.2 - Funções de Transação (EI e EO):
Entradas Externas (EI): Dados que entram no sistema para serem processados ou armazenados (Ex: formulário de cadastro).
Saídas Externas (EO): Dados processados que saem do sistema envolvidos em cálculos ou atualizações (Ex: relatórios gerenciais).

Flashcard 2.3 - Consultas e Dados (EQ, ILF e EIF):
Consultas Externas (EQ): Busca simples de dados exibida na tela sem alteração ou cálculo interno.
Arquivos Lógicos Internos (ILF): Dados mantidos e gerenciados dentro do próprio sistema.
Arquivos de Interface Externa (EIF): Dados usados pelo sistema que pertencem a outra API ou software externo.

Flashcard 2.4 - Como Calcular: O cálculo exige classificar as funções do sistema entre os 5 elementos (EI, EO, EQ, ILF, EIF), aplicar um peso de complexidade para cada um deles (Ex: ILF vale 10, EO vale 5), multiplicar e somar tudo no final.

Flashcard 2.5 - Vantagens e Benefícios: É totalmente independente da linguagem de programação. Pode ser calculado na fase de planejamento, bem antes do código ser escrito, sendo ideal para estimar custos, prazos e esforço.

Flashcard 2.6 - Fórmula e Exemplo de Cálculo na Prática: A fórmula básica consiste em multiplicar a quantidade de elementos identificados pelos seus respectivos pesos de complexidade e somar os resultados: Total PF = (Qtd. EI x Peso) + (Qtd. EO x Peso) + (Qtd. EQ x Peso) + (Qtd. ILF x Peso) + (Qtd. EIF x Peso).

Flashcard 2.7 - Exemplo Prático (Pesos do PDF: EI=4, EO=5, EQ=4, ILF=10): Um pequeno sistema com 1 Entrada (EI), 1 Saída (EO) e 1 Arquivo Interno (ILF) resulta no Cálculo: EI = 1 x 4 = 4, EO = 1 x 5 = 5, ILF = 1 x 10 = 10. Resultado: Total PF = 4 + 5 + 10 = 19 PF. O valor mede o valor gerado para o usuário, e não o tamanho das linhas de código.

Conteúdo dos Flashcards: Complexidade Ciclomática (McCabe)

Flashcard 3.1 - Definição: É uma métrica criada por Thomas McCabe em 1976 para mensurar a complexidade de um módulo (função ou método). Ela funciona contando o número de caminhos linearmente independentes de execução que o código pode seguir até o final.

Flashcard 3.2 - Fatores de Aumento da Complexidade:
A complexidade aumenta a cada Nó-predicado, que são os pontos de decisão e desvio de fluxo no código.
Exemplos práticos que elevam essa métrica são as estruturas condicionais e de repetição, como if, while e switch.

Flashcard 3.3 - Limites de Risco (Valores de Referência):
1 a 10: Método simples (Baixo risco).
11 a 20: Razoavelmente complexo (Moderado risco).
21 a 50: Muito complexo (Elevado risco).
Maior que 50: Altíssimo risco e bastante instável.

Flashcard 3.4 - Fórmulas Equivalentes (Cálculo no Grafo): A partir de um grafo de fluxo de controle, existem três fórmulas equivalentes para achar a complexidade V(G):
1. V(G) = R (Onde R é o número de regiões/espaços fechados no grafo).
2. V(G) = E - N + 2 (Onde E é o número de arestas/setas e N é o número de nós).
3. V(G) = P + 1 (Onde P é o número de nós-predicado/pontos de decisão).

Flashcard 3.5 - Exemplo Prático de Cálculo:
Pontos de Decisão (P): Em um código com um if e um while, temos 2 pontos de decisão (P=2).
Cálculo: Usando a fórmula V(G) = P + 1, o cálculo é V(G) = 2 + 1 = 3.

Flashcard 3.6 - Teste de Cobertura Mínima: O valor da complexidade determina o número mínimo de testes unitários necessários para cobrir todos os caminhos possíveis do código! Se um método possui uma complexidade calculada de 3, isso indica exatamente que são necessários no mínimo 3 testes unitários distintos.

1.4 Conteúdo dos Flashcards: Profundidade de Aninhamento (Nesting Depth)

Flashcard 4.1 - O que mede: Mede o número de níveis de blocos de código que estão colocados um dentro do outro.

Flashcard 4.2 - Quais blocos contam: Praticamente todas as estruturas de controle e desvio contam, incluindo blocos condicionais, laços de repetição e tratamento de exceções.

Flashcard 4.3 - Por que é importante: Porque ela está diretamente ligada à qualidade e legibilidade do código.

Flashcard 4.4 - Exemplo Prático de Cálculo: Imagine este trecho de código:

```
for (let i = 0; i < lista.length; i++) {   // Nível 1
    if (lista[i] > 0) {                     // Nível 2
        if (lista[i] % 2 === 0) {           // Nível 3
            console.log(lista[i]);
        }
    }
}
```

Resultado: A profundidade máxima de aninhamento é 3. Códigos muito aninhados (conhecidos como ”código em formato de pirâmide” ou Arrow Anti-pattern) são péssimos para ler e devem ser refatorados dividindo as funções.

Conteúdo dos Flashcards: Quantidade de Parâmetros por Método

Flashcard 5.1 - O que mede: Ela conta o número de parâmetros formais presentes na assinatura de uma função ou método.

Flashcard 5.2 - Valores de Referência:
0 a 2 parâmetros: Baixa complexidade (Ideal).
3 a 4 parâmetros: Complexidade moderada.
5 ou mais parâmetros: Alta complexidade (Sinal de Code Smell).

Flashcard 5.3 - O que muitos parâmetros indicam: Geralmente indica que a função está fazendo coisas demais.

Flashcard 5.4 - Impacto no Software: O excesso de parâmetros aumenta a dificuldade de entendimento.
Exemplo Prático em Código: A métrica conta as variáveis na assinatura da função. Veja a diferença de complexidade nos exemplos abaixo:

1. Baixa Complexidade (0-2 parâmetros):

```
function salvarUsuario(nome, email) {
  // 2 parâmetros
  // Código limpo e focado
}
```

2. Alta Complexidade (5 ou mais parâmetros):

```
function processarPedido(id, item, qtd, preco, cupom, frete, endereco) {
  // 7 parâmetros!
}
```

Conteúdo dos Flashcards: Índice de Manutenibilidade (Maintainability Index - MI)

Flashcard 6.1 - O que é: É uma métrica composta que combina múltiplos fatores estruturais do código.

Flashcard 6.2 - Composição do Cálculo:
Tamanho do código: medido em Linhas de Código (LOC).
Complexidade estrutural: medida pela Complexidade Ciclomática.
Métricas de Halstead: que avaliam o volume e os operadores do código.

Flashcard 6.3 - Interpretação do Resultado:
Valores Altos: Indicam que o código é limpo, bem estruturado e mais fácil de manter.
Valores Baixos: Servem como um sinal de alerta para códigos potencialmente complexos e difíceis de evoluir ou refatorar.

Flashcard 6.4 - Exemplo Prático de Aplicação:
Exemplo de Painel:
Método A: LOC = 8, Complexidade = 1 $\rightarrow$ MI = 120 (Excelente/Verde)
Método B: LOC = 150, Complexidade = 24 $\rightarrow$ MI = 35 (Crítico/Vermelho)
Ação do Dev: O time bate o olho no score baixo do Método B e sabe exatamente qual trecho do software precisa ser refatorado com urgência para evitar o acúmulo de dívida técnica.

Conteúdo dos Flashcards: Code Smells

Flashcard 7.1 - O que são: São padrões ou características estruturais no código que sugerem problemas latentes de design.

Flashcard 7.2 - O que eles indicam: A presença deles serve como um termômetro para a saúde do software.

Flashcard 7.3 - Exemplos Práticos:
- Funções longas.
- Classes muito grandes (conhecidas como God Class).
- Muitos parâmetros por método.
- Duplicação de código.

Flashcard 7.4 - Exemplo em Código (God Class): É aquela classe inflada que centraliza responsabilidades demais.

```
class GerenciadorDoSistema {
  // Controla usuários, conexões, relatórios, notas fiscais, emails...
  function cadastrarUsuario() { ... }
  function conectarBanco() { ... }
  function gerarPDF() { ... }
  function enviarEmailMkt() { ... }
  function calcularImposto() { ... }
}
```

Flashcard 7.5 - Ação de Refatoração: O desenvolvedor deve aplicar o Princípio de Responsabilidade Única.

Conteúdo dos Flashcards: Indicadores de Churn (Frequência de Mudanças)

Flashcard 8.1 - O que medem: Medem o volume e a quantidade de modificações feitas no código-fonte ao longo do tempo.

Flashcard 8.2 - O que um Churn Alto indica: Pode indicar que o código está instável ou que os requisitos estão mal definidos.

Flashcard 8.3 - Relação com a Manutenibilidade: Sistemas com alta manutenibilidade tendem a apresentar mudanças mais localizadas e seguras.

Flashcard 8.4 - Exemplo Prático de Monitoramento:
`usuario_controller.js`: Alterado 54 vezes no mês (Churn Alto / Instável)
`pagamento_service.js`: Alterado 2 vezes no mês (Churn Baixo / Estável)

Flashcard 8.5 - Ação do Desenvolvedor: O time percebe que o arquivo muda toda hora e decide refatorar.

Conteúdo dos Flashcards: Número de Módulos, Classes, Métodos ou Requisitos

Flashcard 9.1 - O que são: São métricas de tamanho de produto que medem componentes estruturais de alto nível.

Flashcard 9.2 - Número de Requisitos: Mede o tamanho do escopo do software antes mesmo dele ser codificado.

Flashcard 9.3 - Métodos por Classe: Avalia a distribuição da lógica de programação no sistema.

Flashcard 9.4 - Exemplo de Relatório: Número de Classes: 12, Número de Métodos: 48.